OHS REMOTE — CAPSTONE DOCUMENTATION

Capstone347 • CSD-235

Frontend Developer Setup Guide

Environment, Install, Run, Troubleshoot

Capstone347 — CSD-235

April 14, 2026

Contents

1 Who this guide is for **2**

2 Scope **2**

3 Environment requirements **2**
3.1 Software you must install . . . 2
3.2 Node 20 via `nvm` (recommended) . . . 3

4 Accounts and external services **3**

5 Installation **3**
5.1 Step 1 — Clone the frontend repository . . . 3
5.2 Step 2 — Install dependencies . . . 4
5.3 Step 3 — Make sure the backend is running . . . 4
5.4 Step 4 — Start the dev server . . . 4

6 Configuration **4**
6.1 Environment variables . . . 4
6.2 Pointing at a different backend . . . 5

7 Verification checklist **5**

8 Day-to-day commands **5**

9 Build and deployment **6**

10 Troubleshooting **6**
10.1 `EADDRINUSE: address already in use 0.0.0.0:8080` . . . 6
10.2 "Old docs told me port 5173" . . . 6
10.3 `npm install` fails with peer-dependency errors . . . 7
10.4 Every API call 401s and I get bounced to `/login` . . . 7
10.5 `Access to fetch at 'http://localhost:8000/...' blocked by CORS policy` . . . 7
10.6 Admin login fails with credentials I know are correct . . . 7
10.7 Stripe checkout button does nothing . . . 8
10.8 I uploaded a logo and the wizard lost it after a refresh . . . 8
10.9 Step 3 "Generate preview" hangs forever . . . 8
10.10 I changed the base URL but the old URL is still being hit . . . 8
10.11 `npm run build` fails with type errors I do not see in the editor . . . 8
10.12 Tests fail with `ReferenceError: matchMedia is not defined` . . . 9
10.13 ESLint yells about unused vars in my editor . . . 9

11 Where to go next **9**

1 Who this guide is for

This document gets a new developer from a clean laptop to a running OHS Remote frontend talking to a local backend. It covers the exact prerequisites, the install command, how to run the dev server, how to point the frontend at a backend, and a symptom-first troubleshooting section for the failure modes we actually hit during the capstone.

If you are setting up the backend, read `developer_setup_backend.pdf` first — the frontend is useless without a running backend, and most "frontend" bugs during first-run are really backend or CORS issues.

Important

Frontend repository: https://github.com/Capstone347/ohs_frontend

2 Scope

What this guide does cover:

- Installing Node 20 LTS and npm.
- Cloning the frontend repo and installing dependencies.
- Starting the Vite dev server on port 8080.
- Pointing the frontend at a backend base URL.
- Running the lint, test, and build commands.
- Troubleshooting the failure modes documented in the frontend repository's own `TROUBLESHOOTING.md`.

What this guide does *not* cover:

- Production deployment. No production stack is checked in today.
- Styling conventions, component architecture, or how to add a new feature. Those live in `STYLING_GUIDE.md`, `ARCHITECTURE.md`, and `DEVELOPMENT_GUIDE.md` inside the frontend repository's `docs/` folder.
- Backend setup.

3 Environment requirements

3.1 Software you must install

Tool	Version	Why
Node.js	20 LTS	The frontend targets Node 20 LTS. The repo does *not* ship an `.nvmrc` or an `engines` field, so you are responsible for installing it yourself. Older or newer majors may work but are not what we developed against.

Tool	Version	Why
npm	Ships with Node	Canonical package manager. Do not use bun, yarn, or pnpm — a stale `bun.lockb` exists in the repo and should be ignored.
Git	Any recent	Cloning the repository.
A modern browser	Current	Chrome, Edge, or Firefox. The dev workflow assumes you can open DevTools (Console, Network, Application tabs) to debug.

3.2 Node 20 via `nvm` (recommended)

The easiest way to install and switch Node versions is with `nvm`:

- macOS / Linux: https://github.com/nvm-sh/nvm#installing-and-updating
- Windows: https://github.com/coreybutler/nvm-windows#installation--upgrades

Once `nvm` is installed, run:

```
nvm install 20
nvm use 20
node --version  # should print v20.x.x
npm --version
```

If you do not want `nvm`, install Node 20 directly from https://nodejs.org/en/download.

Warning

The repo contains both `package-lock.json` (current) and a stale `bun.lockb`. **Use npm.** Do not run `bun install` — it will produce a broken node_modules that causes confusing peer-dependency errors later.

4 Accounts and external services

Unlike the backend, the frontend needs *zero* external accounts for local development. There are no API keys to configure, no `.env` file, and no third-party SDK registrations. Every external integration (Stripe, OpenAI, SMTP) is reached indirectly through the backend, so those accounts belong to the backend setup guide.

The one value that looks like a secret — the Stripe publishable key used in the checkout flow — is fetched at runtime from the backend via `GET /payments/stripe/config`. The frontend never stores or reads it from an env var.

5 Installation

5.1 Step 1 — Clone the frontend repository

```
git clone https://github.com/Capstone347/ohs_frontend.git
cd ohs_frontend
```

The frontend repository lives at `https://github.com/Capstone347/ohs_frontend`.

5.2 Step 2 — Install dependencies

```
npm install
```

First install takes two to three minutes. Peer-dependency warnings are expected as long as the command exits with code `0`. If it errors out, jump to 10 (troubleshooting — "peer-dependency errors").

5.3 Step 3 — Make sure the backend is running

The frontend is not useful on its own. It expects a REST API at `http://localhost:8000/api/v1`. Start the backend using the instructions in `developer_setup_backend.pdf`, then sanity-check it:

```
curl http://localhost:8000/api/v1/health
```

If that call fails, stop here and fix the backend. None of the frontend flows will work without a healthy backend.

5.4 Step 4 — Start the dev server

```
npm run dev
```

Vite will print something like:

```
VITE v5.4.x ready in ### ms

  Local:  http://localhost:8080/
  Network: http://<your-lan-ip>:8080/
```

Open `http://localhost:8080` in your browser.

> **Important**
>
> The dev server listens on **port 8080**, not Vite's default 5173. This is hardcoded in `vite.config.ts:10`. Older documentation may reference 5173 — that is wrong.

6 Configuration

6.1 Environment variables

There are *no* environment variables required to run the frontend today. The repository ships:

- No `.env` file
- No `.env.example`
- No `VITE_*` variables read in `vite.config.ts` or anywhere in `src/`

This is intentional for the current phase of the project — all the values the frontend would need are either reachable through the backend at runtime (Stripe publishable key) or hardcoded for local development (API base URL).

6.2 Pointing at a different backend

The backend base URL is currently hardcoded in two files. If you need to point the frontend at a non-local backend (a teammate's machine, a staging server, an ngrok tunnel), you must edit both:

File	Line	What it holds
`src/services/api.ts`	1	User-facing API base URL. Default: `http://localhost:8000/api/v1`.
`src/services/adminApi.ts`	3	Admin API base URL. Default: `http://localhost:8000/api/v1/admin`.

Warning

Do **not** commit these edits. They are environment-specific and will break the dev flow for everyone else. A planned migration will move both values into a `VITE_API_URL` env var — see `docs/DEVELOPMENT_GUIDE.md` inside the frontend repo for the target convention.

7 Verification checklist

Run through this list after `npm run dev`. Every step should pass before you consider your setup done.

Step	Check
1	`http://localhost:8080` loads the dark-themed marketing hero page.
2	Click "Sign In" or navigate to `/login`. Enter a test email and click the request-OTP button.
3	Check the backend logs — the OTP code is printed there in dev mode. Copy it into the frontend form.
4	You land on the user dashboard at `/dashboard/orders`. This proves the cookie-session auth is working end to end.
5	Click "Start a new order" or navigate to `/app/step-1`. You should see the wizard shell with a light theme and a step indicator.
6	Open DevTools → Network. Every wizard step should issue requests to `http://localhost:8000/api/v1/...` and receive `200` responses.
7	For admin, navigate to `/admin/login` and sign in with credentials seeded by the backend. You should land on the admin dashboard.

If any step fails, read the troubleshooting section below — it is organized by symptom.

8 Day-to-day commands

All commands below are defined in `package.json:scripts`.

Command	Purpose
`npm run dev`	Start the Vite dev server on `:8080` with HMR.
`npm run build`	Production build into `dist/`.
`npm run build:dev`	Development-mode build with source maps and no minification — useful for debugging a production-only issue.
`npm run preview`	Locally serve whatever is in `dist/` so you can smoke-test a production build.
`npm run lint`	Run ESLint 9 (flat config) across the repo. Treat this as the source of truth for lint errors — editor plugins sometimes disagree.
`npm run test`	Run Vitest once (headless). Currently a placeholder suite.
`npm run test:watch`	Run Vitest in watch mode.
`npx tsc -noEmit`	Type-check the project. There is no `npm run typecheck` script today — add one to your shell history.

9 Build and deployment

The frontend has no production deployment configuration checked in. `npm run build` produces a `dist/` folder containing static assets that can be served by any static file host (Netlify, Vercel, S3 + CloudFront, nginx), but the Stripe and backend base URLs are hardcoded for `localhost:8000`, so a production deploy would first require the planned `VITE_API_URL` migration.

For the capstone submission, the verified deliverable is the dev flow — `npm run dev` against a local backend. Production hardening is a follow-up.

10 Troubleshooting

Organized by symptom, not by cause. Match on what you see in your terminal or browser, then read the fix. If your problem is not here, open DevTools (Console + Network) first — most frontend issues announce themselves there.

10.1 `EADDRINUSE: address already in use 0.0.0.0:8080`

Cause. Something else is already bound to port 8080. `vite.config.ts:10` hardcodes the dev server to 8080 — there is no CLI flag that overrides it cleanly.

Fix.

```
lsof -i :8080      # find the offender
kill -9 <pid>      # free the port
npm run dev        # retry
```

If you absolutely need a different port temporarily, edit `vite.config.ts:10` locally but do not commit the change.

10.2 "Old docs told me port 5173"

Cause. Out-of-date documentation. The actual port is **8080** per `vite.config.ts:10`.

Fix. Open http://localhost:8080. If any other doc still references 5173, it is stale — fix it in the same PR.

10.3 `npm install` fails with peer-dependency errors

Cause. Stale `node_modules` from a different package-manager run — usually because someone ran `bun install` earlier and the leftover lockfile confused things.

Fix.

```
rm -rf node_modules package-lock.json
npm install
```

The `bun.lockb` in the repo is stale. Do *not* run `bun install`. The canonical package manager is npm.

10.4 Every API call 401s and I get bounced to `/login`

Cause. `src/services/api.ts:handleResponse` dispatches an `auth:unauthorized` event on 401. `AuthContext` listens for it and clears the session, which redirects to `/login`. The 401 itself usually has one of four causes:

1. **Backend is not running.** Hit http://localhost:8000/api/v1/health in the browser. If that fails, start the backend.
2. **Session cookie was never set.** Open DevTools → Application → Cookies for `http://localhost:8080`. After OTP verification you should see an `auth_session` cookie. If not, your browser is blocking third-party cookies, or the backend is not setting the cookie for your origin.
3. **CORS mismatch.** See the next subsection.
4. **Session expired.** The backend defaults `auth_session` to a 1-hour max age. Log in again.

10.5 `Access to fetch at 'http://localhost:8000/...' blocked by CORS policy`

Cause. The backend must explicitly allow origin `http://localhost:8080` *and* `Access-Control-Allow-Credentials: true` (required because the frontend sends `credentials: 'include'`).

Fix. This is a backend config issue, not a frontend bug. Open the backend repo and add `http://localhost:8080` to the CORS allow list. You cannot work around this with a wildcard `*` origin — credentials + wildcard is forbidden by the CORS spec.

10.6 Admin login fails with credentials I know are correct

Cause. Admin auth is a separate surface from user auth. It posts to `/api/v1/admin/auth/login` and expects a seeded admin user in the backend database.

Fix. Confirm the seeded admin exists in the backend (see the backend repo's `GETTING_STARTED.md` for the seed script and credentials). Admin auth uses a password, not an OTP — if you are requesting an OTP for an admin account, you are on the wrong login page (`/admin/login` vs `/login`).

10.7 Stripe checkout button does nothing

Cause. The frontend does not embed Stripe Elements. It calls `api.createCheckoutSession(orderId)` and then does `window.location.href = checkout_url`. If nothing happens, it is usually one of: (1) the call failed — check the Network tab, (2) the backend did not return a `checkout_url` — check the response payload, or (3) the backend's Stripe integration is not wired up for your environment (test keys missing, webhook not configured).

Fix. The client side of this is intentionally thin. Fix the backend or its Stripe configuration.

10.8 I uploaded a logo and the wizard lost it after a refresh

Cause. `WizardContext` state is in-memory only. A page refresh wipes everything except fields that were already persisted on the backend (`orderId`, `documentId`).

Fix. Do not refresh mid-wizard. LocalStorage hydration for `WizardContext` is not implemented today.

10.9 Step 3 "Generate preview" hangs forever

Cause. Preview generation is a synchronous backend call that renders a DOCX and returns a PDF. For large orders it takes 10+ seconds. If it hangs longer than 60 seconds, the backend almost certainly errored and the frontend is still waiting.

Fix. Check the backend logs and the browser Network tab. If the request failed, a `toast.error` should appear — if it does not, check the browser console.

10.10 I changed the base URL but the old URL is still being hit

Cause. Browser or Vite caching. Vite normally picks up edits to `src/services/*.ts` instantly, but an active service worker or a stale browser cache can still serve old JS.

Fix.

1. Stop the dev server (`Ctrl+C`).
2. Restart with `npm run dev`.
3. Hard-reload the browser tab (`Cmd+Shift+R` / `Ctrl+Shift+R`).
4. If it still happens: `rm -rf dist node_modules/.vite && npm run dev`.

10.11 `npm run build` fails with type errors I do not see in the editor

Cause. The editor's TypeScript language server may be using a different `tsconfig` than the build. The repo's `tsconfig.json` is intentionally lax (`strictNullChecks: false`, `noImplicitAny: false`), so errors that still survive a build are real.

Fix. Run `npx tsc -noEmit` from the repo root — that is effectively what the build does. Fix the errors it prints.

10.12 Tests fail with `ReferenceError: matchMedia is not defined`

Cause. A component under test calls `window.matchMedia` in jsdom and the polyfill in `src/test/setup.ts` did not register.

Fix. Confirm `vitest.config.ts` still references `setupFiles: ['./src/test/setup.ts']`. If a new test created its own `setupFiles` override, merge the existing polyfill in.

10.13 ESLint yells about unused vars in my editor

Cause. The repo intentionally disables `no-unused-vars` at the ESLint level (`eslint.config.js`), but editor integrations sometimes re-enable it from the default TypeScript rules.

Fix. Trust `npm run lint` as the source of truth. If `npm run lint` passes but your editor still flags unused vars, fix the editor — the repo is correct.

11 Where to go next

Once the dev flow is working, the frontend repo ships its own set of documents that are worth reading in order:

- `docs/ARCHITECTURE.md` — how the app is structured (contexts, routes, services).
- `docs/DEVELOPMENT_GUIDE.md` — conventions for adding a feature, plus the planned `VITE_API_URL` migration.
- `docs/STYLING_GUIDE.md` — Tailwind + shadcn/ui usage, color variables, component patterns.
- `docs/API_INTEGRATION.md` — exactly what the frontend expects from each backend endpoint.
- `docs/TROUBLESHOOTING.md` — the authoritative symptom list (this document reuses its core entries).